

체스에 관한 인공지능과 알고리즘 탐구

1407 박진우

목차

서론

1. 연구의 배경
2. 연구 목적

본론

1. 연구 방법
 - I. 데이터/자료 수집 방법
 - II. 질적, 양적 조사
 - III. 타 보고서 및 논문 활용
 - IV. 사용할 분석 도구
2. 알고리즘 구현
 - I. BFS
 - II. 강화된 BFS
 - III. Minimax
 - A. Minimax만
 - B. Alpha-Beta Pruning 가지치기 사용
3. 데이터 분석

결론

1. 분석결과
 - I. 알고리즘에서의 분석 결과
 - II. 데이터 분석에서의 결과
2. 프로젝트 목표와 결과에 대한 연관
3. 주요 발견 사항
4. 향후 연구 개선

서론

1. 연구의 배경

체스는 3000개가 넘는 오프닝과 차례당 20가지 이상의 많은 경우의 수로 유명해 이 전부터 인공지능의 탐구 주제로 많이 선정되어 왔다. 나는 체스에 대해서 평소에 관심이 있었고, 인공지능 탐구 주제로도 적절하다고 판단되어 체스를 인공지능 면에서 스스로 탐구해보고자 체스 인공지능을 직접 구현하고 탐구해 보게 되었다.

2. 연구 목적

본 연구의 목적은 체스 인공지능에 효율적인 휴리스틱 알고리즘을 적용시키고 그 알고리즘이 적용된 인공지능을 학습시켜 성능을 비교하고, 가장 성능이 좋은 모델을 찾아내기 위함이다.

본론

1. 연구 방법

I. 데이터/자료 수집 방법

데이터 분석을 위한 자료는 인터넷의 chess.com에서 수집하였다.

II. 질적, 양적 조사

chess.com에 있는 Garry Kasparov, Magnus Carlsen 등 유명 그랜드마스터의 경기 기록 PGN데이터 30,000개를 사용하였다. PGN이란 체스 경기에 관한 정보, 체스 보드에 놓여진 기물, 주석 등 체스 경기에 관한 다양한 데이터를 컴퓨터가 처리하기 쉽도록 가공된 형식의 파일이다. 또한 체스와 같은 2인용 게임에 자주 사용되는 알고리즘과 기계학습, 인공지능의 데이터 학습에 대하여 찾아보았다.

III. 타 보고서 및 논문 활용

Minimax algorithm, Alpha-Beta Pruning에 관한 위키피디아와 블로그의 글을 참고하였다.

IV. 사용할 분석 도구

포지션별 가중치를 분석하는 데에 python 내부 모듈인 Stockfish와 Python으로 제작한 PGN파일 변환기를 사용할 것이다.

2. 알고리즘 구현

I. BFS

BFS는 Breadth-First-Search의 약자로, 시작 정점으로부터 가까운 정점을 우선적으로 탐색하는 기법이다. 구현이 간단하고, 모든 경우의 수를 탐색할 수 있다는

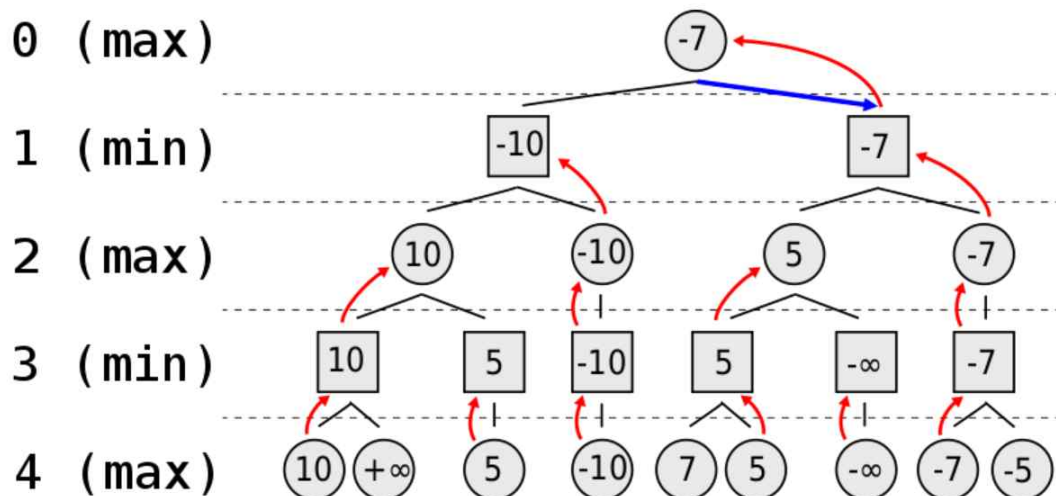
장점이 있어 첫 알고리즘으로 BFS를 선택했다. 총 합계 가중치 / 총 노드 수의 값이 가장 큰 노드를 선택하는 탐색하는 방식으로 구현하였다.

II. 강화된 BFS

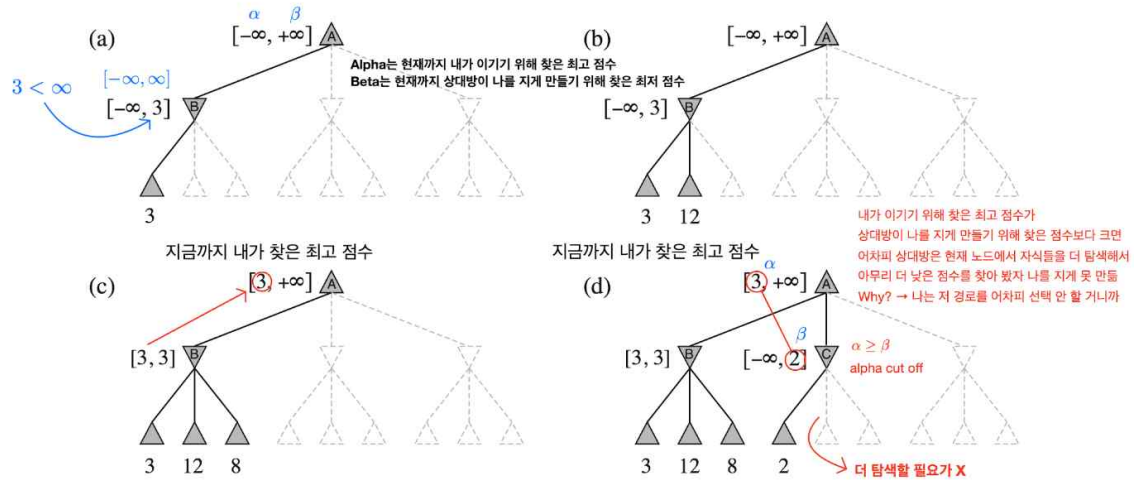
위의 BFS와 같이 총 합계 가중치 / 총 노드 수의 값이 가장 큰 노드를 선택하는 탐색 방식으로 구현하였다. 기물의 가중치를 통해 가지치기하고, 랜덤 함수를 이용해 속도 향상을 노려보았다.

III. Minimax

Minimax 알고리즘은 상대의 최선의 수가 자신에게 최소한의 영향을 미치도록 하는 알고리즘으로, 주로 2인용 게임의 AI에 활용된다. 이 알고리즘은 짝수 깊이(자신의 차례)일 때 최대값, 홀수 깊이(상대방의 차례)일 때 최소값을 넘겨주는 방식으로 최선의 수를 찾는다.



하지만 이 알고리즘은 한 턴당 탐색 횟수가 N일 때 깊이가 D라면 $O(N^D)$ 의 시간복잡도를 가지게 되어 완전탐색과 같은 양의 탐색수가 발생한다. 따라서 추가적으로 Alpha-Beta Pruning이라는 알고리즘을 사용하였다. Alpha-Beta Pruning은 Minimax 탐색에서 최선의 수를 두는 플레이어의 선택에 최종적으로 영향을 미치지 않는 가지를 삭제하는 것이다.



사진에서 볼 수 있는 것처럼 Alpha에는 이기기 하기 위한 최대 가중치, Beta에는 지게하기 위한 최소 가중치가 들어있다. 탐색 중 Alpha보다 Beta가 작은 경우가 발생한다면 더 이상 탐색할 필요가 없다고 판단하여 가지치기한다. 시간복잡도는 Minimax와 같은 $O(N^D)$ 이지만 최적의 수를 빨리 찾은 경우 가지치기가 많이 이루어 연산량이 줄어들게 된다.

위의 두 알고리즘을 사용하여 체스 인공지능을 구현하였다.

A. Minimax만 사용

기물의 가치와 포지션별 가치를 이용해 탐색을 하는 방법으로 구현하였다.

B. Alpha-Beta Pruning 가지치기 사용

기물의 가치와 포지션별 가치를 이용해 탐색 및 가지치기를 하는 방법으로 구현하였다.

3. 데이터 분석

Minimax 탐색을 할 때 사용한 포지션별 가치를 Python으로 만든 학습모델에 기계학습의 일종인 지도학습을 사용하여 구하였다. Stockfish에게 보드의 데이터를 입력시키고 Stockfish가 계산한 승패가중치를 각 기물의 포지션에 적용한 후 평균값을 구하는 방식으로 학습하였다. 그 뒤에는 Stockfish 모듈과의 모의 경기를 통해 성능을 평가 및 비교하였다.

결론

1. 분석 결과

I. 알고리즘에서의 분석 결과

아래의 표는 앞의 알고리즘 구현에서 구현한 4가지의 알고리즘을 적용하였을 때의 탐색노드 수, 소요시간(초), 정확도, 레이팅을 비교한 결과이다. 정확도와 레이팅은

Chess.com의 Komodo25(3200) 인공지능과 경기한 결과를 Chess.com의 리뷰 기능을 사용하여 분석해 예상 레이팅을 얻어낸 결과이다.

	BFS	BFS+가지치기	Minimax	Minimax+AlphaBeta
탐색 노드 수(Depth 4)	197,742	173,592	197,742	64,140
소요시간(초)	21	19	21	7
정확도(%)	56.7	61.7	68.8	70.3
레이팅	400	450	1750	1850

II. 데이터 분석에서의 결과

아래 표는 Minimax + Alpha-Beta 알고리즘을 적용한 체스 AI(Depth 4)의 포지션별 데이터 학습량에 따른 성능의 차이이다.

PGN데이터 학습량	100개	1,000개	10,000개	20,000개	30,000개
정확도(%)	62.2	65.8	69.2	70.3	69.9
레이팅	1100	1300	1800	1850	1800

최고의 모델에서 흰색 폰의 경우 7라인과 8라인, 검은색 폰의 경우 1라인과 2라인, 즉 승진 전 1라인과 승진라인에서 가장 높은 평갯값을 보여주었다. 나이트의 경우, 중앙이 대체로 높았고 특정 위치(c3,c6,f3,f6)에서 높은 평갯값이 측정되었다. 비숍의 경우, 흑색 칸, 백색 칸에 상관 없이 중앙에 위치할수록 평갯값이 높게 측정되었다. 룯과 퀸은 중앙이 다소 높게 측정되었다.

2. 프로젝트 목표와 결과에 대한 연관

프로젝트의 목표는 가장 효율적이고 빠른 체스 모델을 찾는 것이었다.. 위 프로젝트를 통해 효율적이고 빠른 체스 알고리즘을 찾는 데에 성공하였다. 또한 데이터 분석에 인공지능을 사용한 것도 체스에의 인공지능 활용이라는 목표에 부합한다.

3. 주요 발견 사항

가지치기를 적용했을 때 탐색량과 시간이 현저하게 줄어들었다. 특히 Alpha-Beta Prunning을 적용했을 때 속도가 3배 이상 향상되었다. 평가 함수에 포지션별 가중치를 추가했을 때 정확도가 크게 증가했다. 또한 학습 데이터의 양에 따른 정확도는 학습량에 비례하며 증가하다가 일정 수준(10,000개)에 도달했을 때 정확도의 증가량이 감소했고, 30,000개를 학습했을 때 오히려 레이팅이 소폭 감소한 모습을 보였다.

4. 향후 연구 개선점

향후 연구에서는 시간을 더 투자하여 강화학습을 적용한 체스 AI와도 비교해 볼 계획이다. 또한 포지션 뿐만 아니라 공격이 가능한 위치의 개수, 오프닝/미들게임/엔드게임에서의 포지션 비교 등 더 다양한 요소를 적용하여 비교해 볼 것이다.

참고 문헌

<https://glanceyes.com/entry/Adversarial-Search%EC%9D%98-Minimax-Search%EC%99%80-Alpha-beta-Pruning>

https://en.wikipedia.org/wiki/Alpha%E2%80%93beta_pruning

<https://en.wikipedia.org/wiki/Minimax>